Mandatory Access Control and Detection

Computing Systems Security · MCS and MEI · University of Minho · 2026-27

Vítor Francisco Fonte <vff@di.uminho.pt> · 2026-08-19

MAC and Detection: Lecture Overview

What DAC Cannot Do: why ownership stops being an answer, and what a policy above the owner buys

AppArmor in Depth: profiles keyed by path: authoring one, enforcing it, and tightening it until it fits

The Detection Signal: the same denial log read from the defender's chair, where this unit's detection thread starts

Into the Lab: author, enforce, tighten, then read your own denials as an intrusion signal

The tab on the right edge of each slide shows the tier of the slide.

Lecture Core: covered in session. Lab Core: connects to laboratory work. Study Support: post-session review. Reference: lookup material.

What DAC Cannot Do

- Unix permissions are **discretionary**: the owner of a resource decides who may touch it
- So a process running as the owner can give that access away, and the kernel treats it as legitimate
- And root, with the capabilities it normally holds, gets past DAC. There is no further question to ask
- Lab 6 made this concrete: one sudo rule, one shell escape, and ownership stopped meaning anything

Discretionary means the policy is whatever the owner's process decided this instant.

A Policy Above the Owner

- **Mandatory access control**: a system-wide policy fixes what a program may do, whatever UID it runs as
- The owner cannot relax it, and neither can the confined program: that takes policy administration
- So a confined root process is boxed in: it passes the DAC checks and the kernel refuses it anyway
- You will watch exactly that happen in the lab, in one command, on your own machine

Under MAC, who you are does not settle it. The policy says what this program may do.

Two Kinds of Control, Not Two Strengths

- DAC and MAC answer different questions, and neither one replaces the other
- DAC gives per-user ownership, and the everyday sharing that multi-user work runs on
- MAC constrains a program that already holds the identity it needs
- Real systems run both layers, and an operation has to get past each of them independently



Both gates, every time. A permitted operation is one that neither layer refused.

The Reference Monitor, Again

- From Lecture 1, revisited in Lecture 5: complete mediation, tamper resistance, and small enough to verify
- LSM (Linux Security Modules) hooks check each mediated operation in the kernel: no syscall routes around them
- Tamper resistance: a confined process cannot change its own policy unless the profile grants `mac_admin`
- Verifiability is the property you can lose by accident: a profile nobody can read is one nobody has checked

Two of the three properties come from the kernel. The third one you have to earn, by writing less.

Study Support

The Same Idea, Five Systems

- **SELinux**: every object carries a label, and the policy is written about types rather than about paths
- **AppArmor**: profiles keyed by the path of the executable, which is the rest of this hour
- **FreeBSD jails**: a process tree confined to its own filesystem root, network and process view
- **OpenBSD `pledge` and `unveil`**: the program restricts itself and, once locked, cannot lift it
- **Windows** integrity levels and tokens: a lower level cannot write to a higher one; by default, reads are left to the ACL

One idea, five vocabularies: a policy that constrains a process beyond its owner's discretion.

AppArmor in Depth

If the policy is a file somebody writes by hand, what does an honest one look like?

Profiles Keyed by Path

- A profile is usually attached by the **path of its executable**, and loaded into the kernel by `apparmor_parser`
- Inside it, a list of rules: this file may be read, that one written, this library mapped, this capability used
- Whatever AppArmor mediates and the profile does not grant is denied. The user's permissions grant nothing here
- Ubuntu ships profiles for the obvious targets, and `/etc/apparmor.d/` is where they all live

Default deny, one program at a time. DAC is still checked, and the profile lists what the program may touch.

What Is Loaded, and in Which Mode

```
$ sudo aa-status
apparmor module is loaded.
118 profiles are loaded.
24 profiles are in enforce mode.
4 profiles are in complain mode.
90 profiles are in unconfined mode.
1 processes have profiles defined.
   /usr/sbin/rsyslogd (880) rsyslogd
```

- **118 loaded, 24 enforcing, 90 unconfined**: loaded, but confining nothing. The gap is huge
- **One process** on this machine is confined: `rsyslogd`. A stock system covers very little
- `docker-default` is among the 24 enforcing, with no container running: it attaches when one starts
- `/etc/apparmor.d/` ships 109 files, mostly for software not installed here

Loaded is not enforced. Ask is it loaded, then is it enforcing, every time.

Complain First, Then Enforce

- **Complain** allows what the profile does not grant, and logs it. **Enforce** refuses it and logs it too
- For the same file read both modes log the same fields; only `apparmor=` says `ALLOWED` or `DENIED`
- For an unknown service you usually load in complain mode first and read what it needs, before you break it
- Lab 8's profile is two rules long, so you can read it in full and load it straight into enforce

Complain mode is how you find out what a program does. Enforce mode is how you make that answer binding.

A Profile You Can Read in Full

```
# /etc/apparmor.d/usr.local.bin.lab8svc
#include <tunables/global>
/usr/local/bin/lab8svc {
  #include <abstractions/base>
  /usr/local/bin/lab8svc mr,
  /etc/hostname r,
}

$ sudo apparmor_parser -r /etc/apparmor.d/usr.local...
```

- Two rules: read (`r`) and executable-map (`m`) its own binary, read one file. The rest comes from the base
- `abstractions/base` pulls in what every program needs, chiefly the shared libraries the loader maps
- `apparmor_parser -r` loads or reloads it. There is no daemon to restart and no reboot
- For a real service, `aa-genprof` watches the program run and proposes the rules for you

Two rules on top of the base. You can hold all of it in your head, which is the point of doing it by hand.

The Gap You Did Not Know Was There

- Before you run it, predict: which reads does this profile allow, and which does it now forbid?
- Most people under-count: `abstractions/base` also grants locale data, `/dev/urandom` and some of `/proc`
- The distance between what you think a program does and what it does is where profile authoring lives
- An attacker enumerates the same distance, from the other side, and calls it the attack surface

The profile is a written claim about a program's behaviour. Writing it is how you find out the claim was wrong.

Denied, and root Made No Difference

```
$ sudo /usr/local/bin/lab8svc /etc/hostname
ssc-lab

$ sudo /usr/local/bin/lab8svc /etc/shadow; echo exit=$?
/usr/local/bin/lab8svc: /etc/shadow: Permission denied
exit=1
```

- Both commands ran as **root**, so every DAC check passed and the second one was refused anyway
- That is this hour's whole claim, on one screen: the policy overrode the identity
- Run it as an ordinary user and you learn nothing: DAC would have refused it too
- For a real service, `aa-logprof` reads the denials, asks which to allow, and reloads

Run it as root or the demonstration proves nothing. Both layers refusing looks the same as one.

Lab Core

Remove What You Granted

```
# delete the /etc/hostname rule, then reload:
$ sudo apparmor_parser -r /etc/apparmor.d/usr.local...

$ sudo /usr/local/bin/lab8svc /etc/hostname; echo
exit=$?
/usr/local/bin/lab8svc: /etc/hostname: Permission denied
exit=1
```

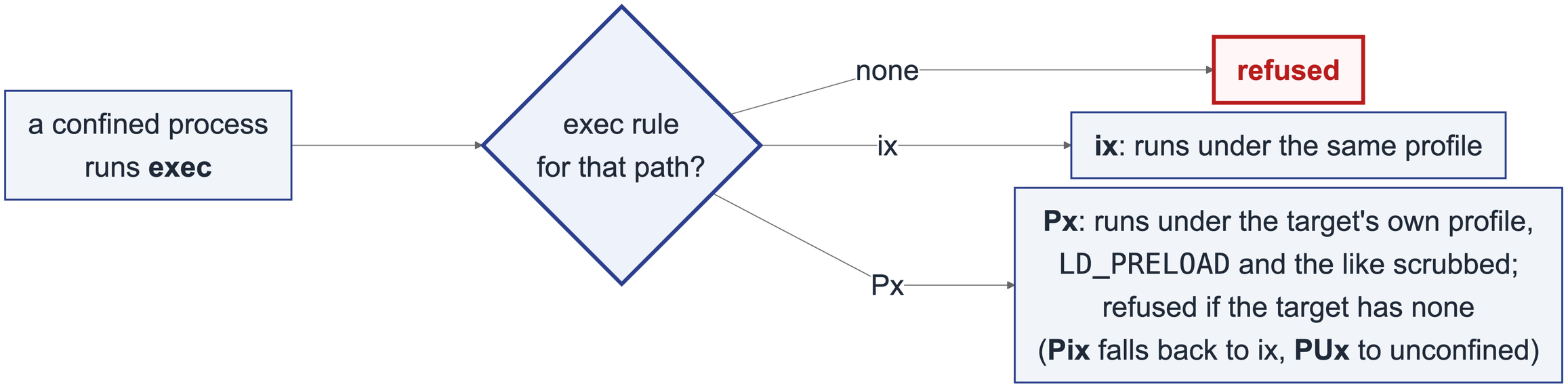
- The read that worked is now refused: that rule alone granted it. In the lab, the rule stays out
- This is the step that separates a competent profile from a lazy one, and it takes one command
- Grant more than the program's job needs, and the surplus is the attacker's
- In a real service, test that it works without the rule

Lab Core

Least privilege is a removal you performed and observed.

And When It Runs Something Else

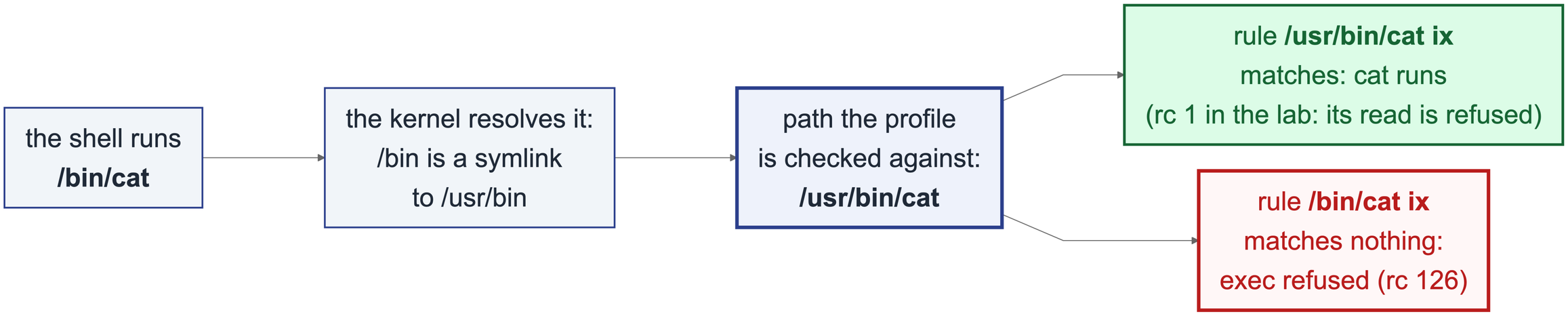
- In this lab a process runs under one profile. When it `execs` a program, an exec rule decides; with none, it is refused
- With `/usr/bin/dash ix` (`/bin/sh` resolves there), whoever reaches the shell can use all the daemon may



A profile that must run a helper is not operationally complete until that transition is chosen and tested.

Path-Based, and What That Costs

- AppArmor keys the profile on the executable's **path**: a copy, rename or hard link is not confined by it
- **SELinux** labels the object instead: the label travels with the inode, so a rename changes nothing
- The trade-off goes both ways: AppArmor profiles are easy to read, and SELinux policy is much harder



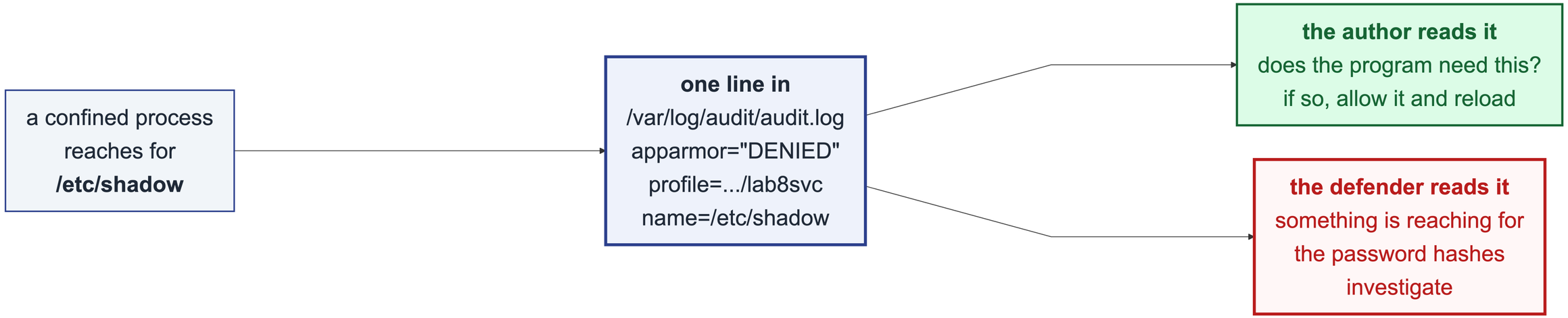
A copy or a hard link is a different program to the policy. A symlink is resolved to the real path first.

The Detection Signal

You have read these denials as the profile's author. What do they mean to somebody who did not cause them?

One Line, Two Readings

- The same entry, unchanged, means two entirely different things depending on who reads it
- To the author: is a rule missing? To a defender: a confined process reaching for the password hashes
- Here prevention and detection share one artefact
- This control hands you a detection feed because default-deny says no aloud



Every denial you cause in the lab is an intrusion signal, whether or not anybody reads it as one.

Reading a Denial as a Defender

```
type=AVC msg=audit(1787067373.628:4565):
  apparmor="DENIED" operation="open" class="file"
  profile="/usr/local/bin/lab8svc"
  name="/etc/shadow"
  pid=20297 comm="lab8svc"
  requested_mask="r" denied_mask="r" fsuid=0 ouid=0
```

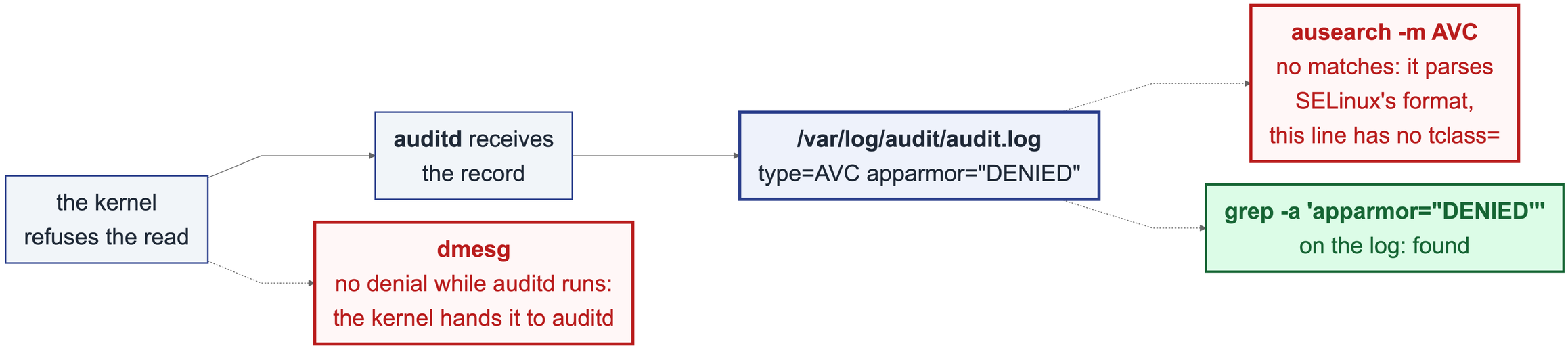
- **Which process**: `comm`, `pid`. **Which policy**: `profile`. **What it wanted**: `operation`, `name`
- The fact: this program, with filesystem UID 0 (`fsuid`), was denied a read of the shadow file
- What it cannot tell you is **why**: an attack, a bug, or a change the profile has not seen
- One line is a question worth asking, not an answer. The answer needs context this record lacks

Read every field. Then say out loud which of your conclusions the line actually supports.

Lab Core

Empty Is Not Evidence of Absence

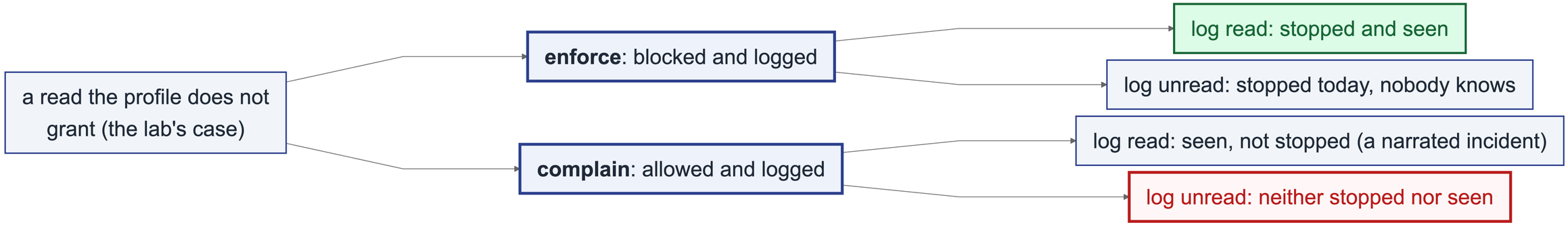
- Neither command a defender reaches for first shows the denial, on this image
- A defender who trusted either of those would report a quiet system while the service was being probed



Verifying that your tool surfaces the signal is the first job of detection.

Prevention and Detection

- Enforcement alone stops today's attempt and tells you nothing, unless somebody reads the log
- A serious posture wants both: a control that refuses, and a log that a human or a SIEM actually reviews



Enforcement without review is blind. Review without enforcement is a narrated incident.

The Thread From Here

- This starts the unit's detection thread, which runs to the capstone
- **Week 11**: the same question asked of a compose stack, where the logs are per-container and plural
- **Week 12**: what an escape attempt looks like from the outside, and which layer recorded it
- **The capstone**: for every step of your attack chain, what a defender watching the host would have seen

Every attack you run from here on gets the same second question: what did it leave behind?

Today in the Laboratory

Snapshot lab8-clean before you start. Confinement is supposed to break things, and you will want the way back.

- Build the target, write the two-rule profile by hand, and load it
- Predict the allowed and denied reads **before** running anything, and write the prediction down
- Show the denial under `sudo`, then remove the rule you granted and confirm that read is refused too
- Then read your own denial as a defender, including the two commands that will not show it to you

This produces Lab report 3, and its detection half is weighted in the rubric.

Lab Core

Block 3 in One Slide

- **Week 6**: privilege is an identity, it is granted whole, and it escapes through whatever it is allowed to run
- **Week 7**: privilege is a list, so grant one item instead of forty. That one item still works in full
- **Week 8**: a policy above the owner refuses what identity would have permitted, and says so in a log
- Three weeks, one direction of travel: from who you are, to what you hold, to what you are permitted to do

Lab report 3 writes up Lab 8 and carries all three labs' reflections: build them on this through-line.

Reading and Reference, All Optional

- **apparmor(7) and apparmor.d(5)**. The model and the profile syntax. Between them they cover everything in today's laboratory. https://manpages.ubuntu.com/manpages/noble/man7/apparmor.7.html
- **aa-genprof(8), aa-logprof(8), aa-complain, aa-enforce**. The assisted authoring route, for a service too large to write by hand.
- **auditd(8) and ausearch(8)**. Where the signal is written, and the query that misses it on this image.
- **SELinux, type enforcement**. The label-based alternative, and the contrast Lab 8's second reflection question asks about. https://github.com/SELinuxProject/selinux-notebook/blob/main/src/type_enforcement.md
- **W. Du, Computer Security: A Hands-on Approach, 3rd ed., 2022**. The mandatory access control and SELinux material.

Reference

Key Takeaways

- MAC is policy the owner cannot change, DAC is the owner's to give away, and a real system needs both gates
- A profile is a written claim about what a program does. Authoring one is how you discover the claim was wrong
- **Least privilege is a removal you performed** and observed
- One denial log, two readings: the author's question about a rule and the defender's intrusion signal
- **Empty output is not evidence of absence.** Prove your detection query catches the thing it claims to catch

Which program, permitted to do what, and would you have seen it if it tried something else?

Reading the Diagrams

- **Slide 5**. A read passes two gates in series: DAC lets root read /etc/shadow through the owner bits, then the profile, which has no rule for it, refuses.
- **Slide 16**. What decides an exec from a confined process: no rule refuses it, ix keeps the same profile, Px moves to the target's own profile or refuses.
- **Slide 17**. On this image the kernel resolves /bin/cat to /usr/bin/cat before the profile is checked: a rule for /usr/bin/cat matches, one for /bin/cat never does.
- **Slide 19**. One denial line: the author asks whether the program needs it, a defender reads a sign of intrusion.
- **Slide 21**. The denial goes through auditd into audit.log; ausearch and dmesg miss it here, and grep -a finds it.
- **Slide 22**. For a read the profile does not grant: only enforce, with a log somebody reads, both stops and sees it.

Reference